

From AI-found bugs to installed protection

Eric Buess | One-page approval draft 1.3 | September 13, 2026

Thank you, Logan, for offering to take a short document to the team. I read the thread as a call for a practical lab-backed effort to make a large share of systems harder to compromise while alignment remains unresolved. A lab could put capped compute and engineering behind shared repairs and owner-authorized deployment, then verify their reach. A pilot could test that delivery mechanism; it would not establish global coverage or settle alignment.

The lab-level proposal. Dedicate a capped defensive-compute and engineering budget to three connected tasks: improve widely shared software and defaults; help owners safely install those changes; and verify adoption and remaining attack paths. Work through maintainers, package and platform releases, cloud providers and managed fleets. That distribution is the proposed route to broad reach. Build on existing initiatives such as Glasswing and AI-assisted browser security work.

Choose channels by reachable exposure, security importance, update authority and observed adoption. Fund the limiting stage-discovery, review, testing or rollout-rather than maximizing findings. Pair donated cycles with maintainer/operator support. Supported firmware and infrastructure configuration fit this approach; hardware redesign and unsupported devices need separate paths.

Start with a 90-day pilot. With a consenting maintainer and operator, select one repeatable web-service deployment class. Finalize permissions, baselines and stop conditions; assemble 20-40 replayable feasibility tasks, including failures and recovery; then run limited canaries and independent review. Expand only if results justify it.

For supported, previously reviewed upgrades and mitigations, target verified canary improvement within one hour of admission. Novel or high-consequence changes take an engineering path. Also measure submission-to-full-deployment time: an hour-long canary is an intermediate milestone, not the full owner experience you described. Retain rejected cases, delays, regressions and failures. Define disruption and follow-up criteria with the operator; successful rollback does not erase an outage.

What would count. Compare against the operator's normal process under declared budgets. Measure addressed exposures, adoption, time, operator effort, regressions and recovery. Report verified protection for specified attack paths over a declared period, not that every patched system is secure. Deduplicate installed populations, retain unknowns and distinguish current attacker tests from future capabilities. There is no defensible global percentage yet.

Keep patch authorship, verification and deployment authority separate. Limit credentials, stage rollouts in small cohorts, and test whether a compromised defender or control channel could spread a harmful change. Preserve an operator-controlled stop and recovery path.

Resources. An illustrative 90-day envelope is 8-12 full-time-equivalent staff and approximately \$0.7-\$1.6 million, including infrastructure and review. These are planning assumptions; approve a capped first phase after partner scoping. They do not price a global rollout.

I developed this proposal with Titus, the coordination workflow I am building, using research and critiques from several model families. Its review record includes limitations; whether the process improves on a strong single model remains to be tested. Open-weight participation is still to be added.

Which distribution partner and deployment class would make the most useful first test? I would welcome corrections and pointers to work we should build on.

Full proposal and review record accompany this draft. No pilot, partnership, funding or endorsement is claimed.

From AI-found bugs to installed protection

A practical response to Logan Graham's challenge

Eric Buess · Approval draft 1.3 · September 13, 2026

Thank you, Logan, for offering to take a short document to the team. I read your challenge as asking how a frontier lab could make a large share of systems harder to compromise while alignment remains unresolved, and your follow-up as asking for a path toward rapid, tested, deployed improvement for owners without disruption. I offer this as a starting point for that discussion, with respect for the work already underway.

Recommendation: a frontier lab could dedicate a capped defensive-compute and engineering budget to a program that connects widely deployed software to the channels that update it. Pair reusable upstream repairs and safer defaults with an owner-authorized service for testing, deploying and verifying them. A 90-day pilot would test that delivery mechanism; the broader program would expand through maintainers, platform vendors and operators with measurable installed reach.

This responds to [Logan's challenge](#) and [OmegaTir's proposal](#) to devote some lab computation to hardening internet, software and hardware systems while alignment remains unresolved. The preceding [Liv Boeree reply](#) makes the misalignment concern explicit, within the discussion prompted by [Dario Amodè's pacing essay](#). Logan's [follow-up](#) imagines an owner submitting a system and getting rapid, tested, deployed improvement without disruption. That is a useful product aspiration, not a security guarantee or demonstrated service level. Hardening complements alignment research, oversight and decisions about capability release. It cannot establish safety against an arbitrarily stronger future agent. I do not claim this pilot reaches a large share of the world's systems; it would produce evidence about whether broader reach can be earned.

In this proposal: [Defensive workflow](#) · [Distribution](#) · [90-day pilot](#) · [Measurement](#) · [Defender accountability](#) · [Titus and alignment research](#) · [Optional technical annex](#)

1. Build the complete defensive path

For an owner-approved deployment, the service inventories relevant versions, dependencies, exposure, identities and permitted agent actions. It establishes functional and security checks, proposes a repair or mitigation, tests it in an isolated replica, obtains the required approval, deploys a canary and measures whether the change actually protects the intended instances. It verifies rollback and recovery before expanding rollout.

Two paths keep this useful without pretending every problem is solved in an hour:

- **Fast path:** supported upgrades, configuration changes and already-reviewed mitigations within a policy the operator approved in advance. Freeze eligibility before the trial. Measure from timestamped admission to verified protection on the canary cohort with rollback tested, against a one-hour target. Count all admitted cases; exceptions, engineering reroutes and regressions within a predeclared

observation window count as misses. Report submission-to-admission delay and full-rollout time separately, retaining rejected submissions. One hour is a target to evaluate, not a promised service level.

- **Engineering path:** novel vulnerabilities, changes to critical dependencies, uncertain compatibility and high-consequence systems. The service prepares evidence and a candidate patch; maintainers and operators retain acceptance and deployment authority. A one-hour diagnosis may be possible; a one-hour safe repair is not promised.

Keep the full owner experience visible. Also measure from an owner's original submission to verified improvement across the entire submitted, eligible deployment, including onboarding, approval and rollout. A one-hour canary after admission is an intermediate milestone for the lab and operator, not success on the broader owner experience you described. An hour-long canary cannot establish lasting security. Before the trial, define functional, availability, data-integrity and latency criteria and the follow-up period with the operator. An unplanned outage or data-integrity failure counts against success even if rollback works; recovery is not the same as avoiding disruption.

The lab provides model access, engineering, isolated evaluation and funding. Maintainers control upstream acceptance; operators control their fleets. An independent evaluator checks a sample of accepted and rejected results against underlying evidence. Generated text and model agreement do not authorize a deployment.

2. The lab-scale strategy: shared components and distribution

The route to a large share of systems is to improve components and defaults that many deployments share, then work through organizations already able to ship those changes. A lab does not need direct administrative access to every device. The first program should build on existing defensive initiatives rather than establish a new institution before learning what is missing.

Workstream	What the lab funds or supplies	How improvements reach systems
Shared software and reusable defenses	Reproduction, candidate repairs, compatibility tests, maintainer support and work on recurring vulnerability classes	Maintainers and package, container, operating-system or browser release channels
Owner-facing hardening	Inventory and policy adapters, isolated validation, canary/recovery tooling, and integration support	Cloud and managed-service providers or endpoint-management operators, within each owner's authorization
Internet and hardware-dependent systems	Supported configuration changes and vendor-approved firmware work, with specialist validation	Infrastructure operators and equipment vendors; physical redesign and unsupported devices require separate, longer paths

These are proposed partner types, not commitments. Begin with one repeatable web-service class for integration, while mapping which shared dependencies and update channels could produce broader reach. Before expansion, obtain evidence of each candidate channel's active installations, supported versions, update authority and adoption rate. Select the next channel by reachable exposure, the

importance of the affected attack paths, and the practical ability to ship and maintain a change. A popular dependency without downstream adoption is not installed protection.

Treat the proposed donated cycles as one resource in that chain. The lab should name a program owner, cap the first-phase compute and spending allocation, and support maintainer/operator time alongside inference. The pilot budget below is a starting envelope; it is not a recommended percentage of a lab's total compute. Reallocate later capacity using measured progress and costs at each stage: if validated repairs are waiting for review or rollout, fund that constraint before generating a larger findings backlog. If discovery is limiting progress in a supported class, additional defensive model work may be appropriate. Preserve separate accounting for alignment research, and publish the resulting allocation and its reasons.

For each accepted change, produce a reusable evidence package: affected versions and assumptions, the candidate repair or configuration, tests, known limitations, deployment instructions and recovery procedure. Distribute it through the maintainer's or operator's existing trusted channel. Local acceptance still matters; the same patch can behave differently under another configuration. The repeated cycle is shared improvement, owner-approved delivery, observed adoption and a new check against the remaining attack paths.

The research portfolio should include current vulnerable versions, insecure configurations, credential exposure and excessive agent permissions. Rank those cases by exposure, default reachability and the impact of the reachable attack path. [Justin Elze's response](#) makes a useful case for that ordering; it should be tested against the operator's actual environment. In parallel, assess a few reusable class-level defenses such as safer parsing components, reduced privilege or stronger build verification. Use testing sanitizers in testing; production mitigations require their own compatibility, security and performance assessment. Assume some exploitable weaknesses will remain despite repair, as [Dino Dai Zovi's response](#) emphasizes. Least privilege, isolation and recovery therefore belong in the initial design. For unsupported or abandoned systems, offer isolation, replacement or retirement options. A network filter is not a universal repair.

This would build on existing work. Anthropic's [Glasswing update](#) documents downstream verification and remediation constraints, and its [expansion](#) describes a planned shift toward disclosing, fixing and deploying patched software. Google's [Chrome account](#) reports an AI-assisted workflow with developer review, with user-update delays remaining a gap. Neither example proves which investment is best in every environment. Instrument each stage and fund the observed constraint, whether compute, review, testing or deployment. [ARIA's Safeguarded AI programme](#) is complementary prior work on mathematical assurance and AI-enabled formal methods for cybersecurity. Its tools may inform bounded components of this pilot; that does not establish the security of an entire deployed system.

3. A 90-day program a lab can decide to fund

The clock starts once a consenting operator, a maintainer and one deployment class are scoped. The first phase finalizes those arrangements before scanning or changing systems.

Period	Accountable owner	Deliverable and decision
Days 1-15	Lab program lead with maintainer and operator leads	Finalize partner agreements, permissions, liability, data handling, baseline workflow, rollback, disclosure and costs. Specify retention, permitted training use, IP access and processor exceptions; reconcile confidentiality requirements with necessary audit evidence. Register all offered systems, including exclusions. If authority or evidence access is inadequate, change scope before scanning. Pause the clock if agreements remain unsigned; do not compress later testing.
Days 16-45	Engineering lead and independent evaluation lead	Build the supported fast path using existing tools. Assemble 20-40 representative replayable tasks, explicitly a feasibility sample rather than a prevalence survey. Include failed repairs, permission refusals, bad fixes and recovery exercises. Freeze comparison rules before scoring.
Days 46-75	Operator release lead	Run a limited canary program on consenting deployments. Compare the service with the operator's normal process under declared budgets, using random assignment or matched cases where feasible. Keep unmatched results separate. Pause rollout after an authority violation, failed recovery or serious regression pending investigation.
Days 76-90	Independent evaluator and program sponsor	Audit outcomes and adverse cases, publish scoped aggregate results, and decide whether to expand, revise or stop. Report exclusions and missing evidence. A successful demonstration earns another deployment class, not a claim of worldwide security.

Illustrative planning envelope: 8-12 full-time-equivalent staff for 90 days across integration, maintainer/operator support, evaluation and program/security work. At an assumed loaded annual cost of \$250,000-\$400,000 each, labor is approximately \$0.49-\$1.18 million. Add a separate \$0.2-\$0.4 million planning allowance for inference, test infrastructure, partner reimbursement and outside review: approximately \$0.7-\$1.6 million total. These are assumptions for discussion, not vendor quotes or funding we have received. Existing staff and infrastructure may reduce incremental cost; a sponsor should approve a capped first phase after partner scoping. Staff the pilot without drawing down alignment research.

4. What would count as progress?

The pilot tests present AI-assisted attackers within recorded tools, access and resource budgets. It can inform preparation for the autonomous misaligned agents motivating this discussion; it cannot measure protection against hypothetical future capabilities.

Publish the admission and outcome funnel. Keep these denominators separate:

- 1. Reach:** all owner-offered systems; the subset eligible for the supported service; and all actual deployments receiving a verified change. Unsupported and unobservable systems remain explicit.
- 2. Time and adoption:** for the fast path, admission to verified canary protection as defined above, plus submission delays and full-rollout time. For the engineering path, validated finding to installed protection. Retain failures and unresolved cases. Report one-hour successes among all admitted

fast-path cases, including subsequent disqualifying regressions.

3. **Security outcomes:** validated attack-path successes against isolated replicas or explicitly authorized targets divided by predeclared test attempts, with model, harness, tools, version, information access and budget fixed and recorded. Report patch bypasses, new privilege paths and failed checks. Independently verify a sample; zero observed successes in a finite test is not proof of security.
4. **Operational cost:** operator/maintainer time, inference cost, regressions, availability impact, rollback and recovery results. A faster insecure change is a failure.

Keep a fixed attacker configuration for comparison, and a separate challenge set using newly available capabilities. Report each available model-and-harness configuration separately, including differences in safeguards and access; do not pool current and stronger configurations into one success rate. A hypothetical future attacker that cannot be tested remains an explicit uncertainty. Do not move the benchmark silently as models change. Set minimum useful improvement and acceptable harm thresholds with operators before the trial; report uncertainty and sample limits. Scale only with supported improvement over the baseline, acceptable operational outcomes, sufficient review capacity and repeatable deployment evidence.

For broader reach, map each distribution channel's observable installed base and deduplicate overlapping deployments. Count a deployment as protected only against the specified exposure or attack path, under recorded configuration and threat assumptions, with current verification over a declared observation period. An installed patch is not whole-system security, and stale or missing verification must be reported separately. Report a defensible estimate for that named population and the unknown remainder. A list of popular libraries, a KEV share or a sum of overlapping download counts cannot establish a percentage of the world's systems. System coverage and risk reduction are different claims.

Flag replay cases whose fixes were public before the model's documented cutoff, and report contamination uncertainty. State who selects and pays the evaluator, their access to underlying evidence and any conflicts; several models agreeing does not establish evaluator independence.

5. Keep the defenders accountable

Run defensive agents with scoped credentials and bounded network access. Store audit events outside their write authority; a missing or corrupted receipt prevents acceptance. Separate the patch author, verifier and deployment authority. Rehearse recovery and credential revocation. Use coordinated, time-bounded disclosure with escalation and active-exploitation exceptions, distinguishing maintainer notification, public advisories and detailed exploit disclosure. Do not publish an exploitable scoreboard or raw customer environments.

Wide distribution also creates correlated failure risk: a mistaken or compromised defender could spread a bad change. Keep tenant credentials and deployment authority separate, use independent validation and small staged cohorts, and prevent the authoring model from changing the release policy or approving its own work. Retain an operator-controlled stop and recovery path that does not depend on the same model or control service. Test malicious artifacts, shared-dependency regressions and compromise of the update/control channel before expansion. A signature establishes an artifact's asserted origin; it does not establish that the change is safe.

When a defensive agent violates intent or conceals failure, preserve the incident and investigate before treating a patch to its prompt as remediation. METR's framework is a useful reference for independent investigation and its evidence limits. The pilot's public report should state its access, scope, omissions and redactions. I do not claim a METR relationship.

6. How Titus helped, and what remains to be tested

I am developing **Titus**, an agent coordination workflow that brings models from different providers into research and critique rounds, retains source-linked contributions and tracks unresolved objections. This proposal began with Grok research and was revised through documented critiques from several model families. The accompanying review note identifies the contributors, incomplete attempts and limits of the process. I use Titus to organize this work; it does not yet operate as a fully autonomous research team.

The useful contribution to test is whether this process catches more consequential errors or produces better validated repairs than a strong single-model workflow at comparable resources. Cross-provider agreement alone is not independent verification. Open-weight frontier participation is still to be added. My proposed ASI.contractors contribution model explores a related idea: contributors receive bounded tasks, and credit or compensation depends on independently checked acceptance. That remains a design concept, not an operating marketplace or a partner in this proposal.

My broader **Alignment Hypothesis** project asks whether developmental environments with persistent consequences, relationships and correction can support useful alignment mechanisms. Its immediate work concerns a measurement instrument and a proposed frozen-model reporting/correction pilot; no developmental learning result has been demonstrated. This hardening proposal addresses a related deployment concern. It neither validates the training ideas nor claims to solve alignment.

7. Optional technical annex: practical questions

Where would this run, and what would an owner have to hand over?

Start with a consenting operator's own environment or an isolated replica it controls, whether on premises or in its cloud account. Begin with a bounded inventory and explicit permission for the selected test and deployment class. Proprietary source, customer data, secrets and unrestricted administrative access are not default inputs. If a task requires additional access, scope it with the owner before proceeding. Some systems will be unsuitable for the pilot; record the exclusion instead of weakening the access boundary. No particular cloud vendor is required by this proposal.

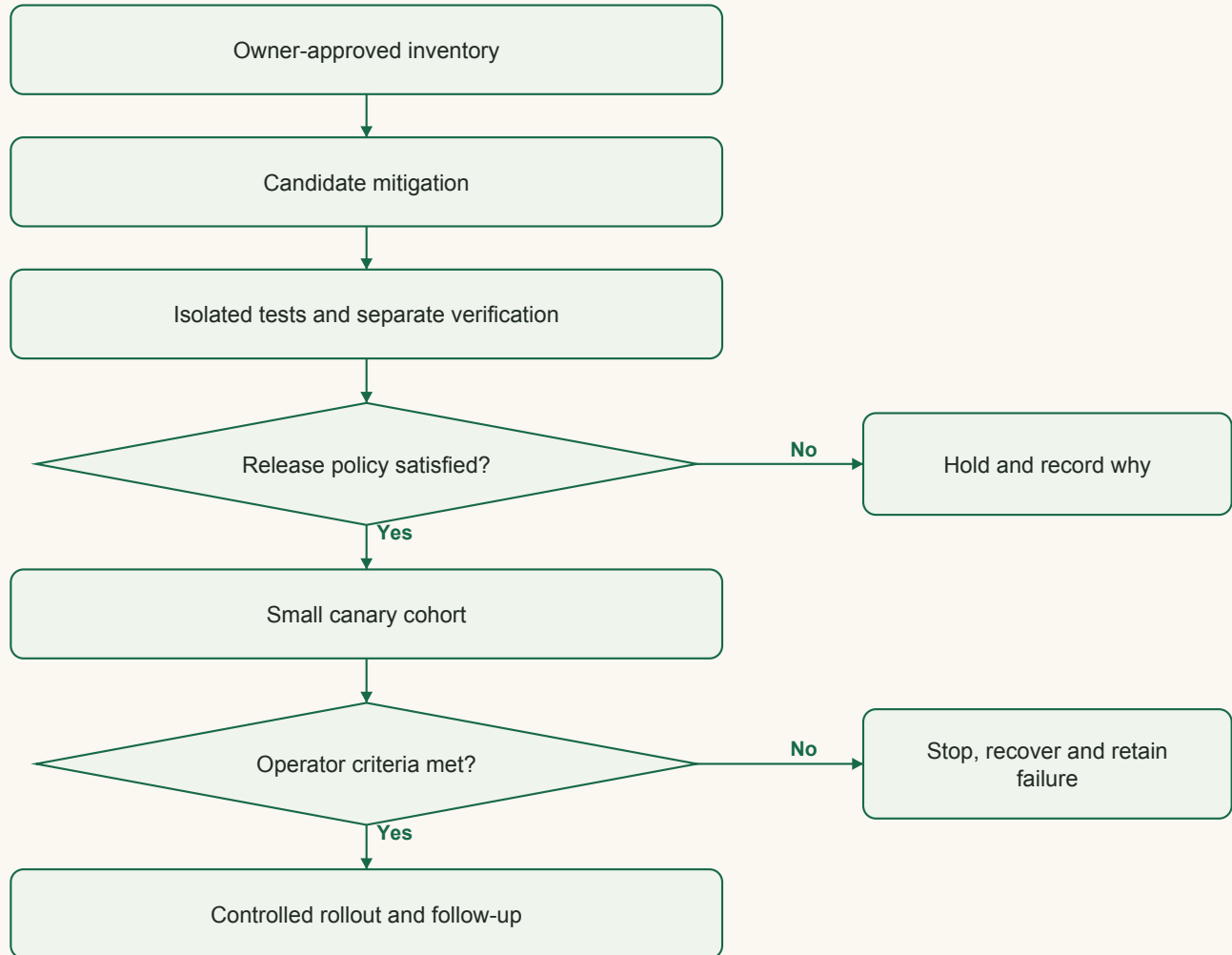
What would the AI actually do?

Models could help interpret permitted inventories, identify candidate mitigations, prepare patches and tests, and compare expected with observed behavior. Separate technical checks verify the candidate and its evidence. An accountable independent evaluator audits a sample of accepted and rejected outcomes; a second model, by itself, does not establish that independence. Ordinary software enforces the approved scope, records events and applies release gates; generated prose or a model's confidence does not grant deployment authority. Fast-path releases follow an operator-approved policy, while novel or consequential changes require the engineering path. This division is proposed, not a claim that a model can safely administer every system unattended.

Could this be a reusable workflow or recipe?

Yes. The first deliverable could be a narrow adapter for one deployment class, with an inventory schema, explicit policy, a candidate change, reproducible tests, an evidence manifest and a deployment/recovery receipt. Keep the common contract small; add environment-specific adapters after their own tests. The following is an illustrative sequence, not an implemented product:

ILLUSTRATIVE WORKFLOW - NOT AN IMPLEMENTED PRODUCT



The record should make a failed test, missing evidence, refusal and rollback as visible as a successful change. Portability comes from tested interfaces and adapters, not copying an agent's entire private configuration.

Would open-weight models help?

They are candidates for work that benefits from local execution, reproducible checkpoints or access to model internals. Closed models may be useful where their measured capabilities justify their use and the data policy permits it. Placement and task fit need evaluation; neither a model's openness nor local hosting establishes security. Open-weight participation has not yet been demonstrated in the review process described here.

Does this advance mechanistic interpretability or alignment training?

It could provide well-documented incidents, counterfactual tests and failure cases for researchers to investigate. Explaining internal mechanisms requires suitable access and additional methods; an external success/failure test does not supply that explanation. A corrected answer or blocked action also does not demonstrate a lasting change in a model's propensity. Any later training use needs explicit data rights, suitable controls and held-out evaluation. These are possible research connections, not results of this proposed pilot.

Is this a replacement for internet protocols or a proposal for a new security firm?

The immediate proposal improves existing software, defaults and deployment channels. It does not depend on replacing TCP/IP or building a new internet. A lab could support existing maintainers, operators and evaluators rather than create a new institution. My separate ASI.contractors ideas concern incentives for independently checked contributions; no such marketplace is assumed here.

How would incentives and future model advances be handled?

Reward installed, independently checked improvements and honest reporting of adverse results, rather than the number of findings or patches. Fund maintainer and operator participation so early partners are not left with uncompensated integration work. Publish who paid for evaluation and any conflicts. Government or other sponsors could support independent evaluation or difficult-to-serve systems, subject to their actual programmes; no funding route or commitment is claimed. Keep a fixed comparison configuration and separately report challenges from newer models. Update threat assumptions and revalidate relevant defenses when capabilities or dependencies change; do not silently rewrite an earlier score.

For the team discussion: A one-page brief accompanies this memo, following your [offer to consider a short document](#). Which supported deployment class and existing partner would make the most informative pilot? What evidence would your team need before expanding it? I would welcome corrections, especially where this duplicates work already in progress or overlooks a practical constraint.

Approval draft 1.3. This is a proposal, not an operating service. No pilot has run, no partner has agreed, and no funding or endorsement is claimed. The public Gist remains version 1.1 until Eric approves this revision.

How this proposal was developed

Eric Buess | September 13, 2026 | Review record for approval draft 1.3

I developed this proposal with Titus, the coordination workflow I am building. Grok supplied the initial research draft. Other models critiqued a frozen source packet independently, followed by an editor's synthesis and a bounded round of cross-review. Source checking and editorial judgment determined which suggestions were retained.

Model or route	Harness and requested setting	Observed contribution
OpenAI / GPT-6 Astra requested	Codex, Ultra requested	Coordinating editor: context recovery, primary-source checks, synthesis and critique dispositions. Not a blind additional reviewer.
Grok 4.6 reported	Grok web app, Heavy selected	Initial research draft, context revision, cross-review and a later bounded search of other public responses in the same conversation.
Gemini 3.1 Pro High reported	Google Antigravity CLI, High requested	Independent initial critique and cross-review.
Gemini 3.8 Flash High reported	Google Antigravity CLI, High requested	Independent initial critique and cross-review. This is another Google model, not another provider.
Muse Spark 1.3 reported	Meta Muse CLI, Ultra requested	Independent initial critique and cross-review, with source-receipt limitations below.
SWE-2 Max reported	Devin CLI, Max model variant	Independent initial critique and cross-review.
Claude Opus 5 reported	Claude Code, Max requested	Final-candidate review completed. An earlier attempt ended after 15 minutes without a final answer and is not counted.
Claude Fable 5.1 Max reported	Devin CLI, Max model variant	Separate final-candidate review completed. This is another Anthropic model, not another provider.

These labels come from native interfaces or local request/response records. They are not provider-signed attestations of backend reasoning effort or effective context size. No open-weight frontier model has participated. The contributors did not all see identical conversational context: Grok retained its earlier conversation, and the editor saw the full process.

What changed

The early draft overstated several things. The revision removes an invalid Glasswing patch-rate comparison, an unsupported causal inference about Chrome, a production recommendation for a testing sanitizer, and unmeasured global-coverage claims. It replaces assumed merge authority and indefinite disclosure delays with operator/maintainer control and bounded disclosure. Staffing and cost figures are now explicit planning assumptions.

Cross-review also found a wording contradiction about quotations and budgets. A further primary-source check showed that Glasswing does describe a planned shift toward repair and deployment support; the final wording acknowledges that plan without implying an already operating autonomous service. The proposal now explicitly treats the one-hour goal as a target and reports different attacker configurations separately. Opus and Fable identified missing context and defender controls in the brief. Opus also required a fixed fast-path clock and failure denominator. Those changes are incorporated, with admission delays and full-rollout time reported separately.

Some reviewers introduced new unsupported numbers or stronger claims. Those were not adopted. Agreement among models was not treated as source verification.

What this process does not establish

Source packets and returned answers were retained with hashes. That helps detect delivery and version errors; it does not prove comprehension. Meta's answers missed the requested terminal source marker, despite a matching recorded input, and Flash's initial answer exceeded its word limit. Those shortcomings remain in the review record. Grok's final cross-review initially rendered blank; reloading the page recovered the answer without another model call.

Opus and Fable reviewed a later frozen brief, memo and source summaries, not peer finals, the outreach draft or this methods note. Neither fetched linked pages. Both returned “send after corrections”; the coordinating editor applied and checked the changes. Fable nevertheless referred to earlier reviewers' positions it had not received; that attribution was not credited. Opus exceeded its requested 1,000-word limit by 20 words. Its harness reported two turns despite a one-turn request, and ancillary Haiku usage; this is a recorded control failure, not an additional substantive review. No private reasoning transcripts are published.

This was a document-development exercise, not a controlled evaluation of model collaboration. There were no blinded outcome judges, matched single-model comparison, deployed pilot or security-effectiveness results. Testing whether this workflow adds value at comparable resources remains future work. The accompanying proposal should stand or fall on its evidence and practical merits, not the number of models listed here.

Update after reviewing other responses

For version 1.1, Eric asked for a check of other responses to Logan. One additional Grok web pass, with Heavy selected, compared all three published version 1.0 files with a limited sample of replies and quotations. The interface displayed 1 minute 28 seconds of thinking; the final response contained 1,038 words. The exact backend model and thinking effort were not attested for this pass.

Subsequent primary-source checks supported crediting Justin Elze's exposure/default-reachability recommendation and Dino Dai Zovi's residual-vulnerability design assumption, and citing ARIA's existing formal-assurance programme. We added explicit data-handling admission requirements as our own design recommendation. Longer posts and attached images were not completely recovered; specific unverified attributions about data retention, authenticity and swarm observatories were omitted. No exhaustive search or novelty claim is made, and these additions have not received another full cross-model round or empirical validation.

Goal-fit review for approval draft 1.2

Eric requested another check against the full post chain. The editor fetched the live version 1.1 Gist and rechecked the official X quotation/reply edges: Dario's announcement, Elon's quotation, Derek Thompson's quotation, Liv Boeree's reply, OmegaTir's reply and Logan's challenge. Logan's later owner-experience and short-document replies were also checked. X truncated Dario's and Derek's longer announcement text; the full Dario essay was recovered from its primary site. No claim is made that the complete Derek post or every descendant reply was recovered.

A separate internal OpenAI reviewer audited the same live files for goal fit, arithmetic and overclaims. This is not another independent provider or a new complete cross-model round. The editor adopted four substantive changes: an explicit lab-scale distribution and resource-allocation strategy; separate canary and end-to-end timing; protection tied to specified exposures and a validity period; and controls for correlated defender or update-channel failures. The staffing arithmetic was checked. These are proposed design improvements, not evidence that the program will meet its targets.

Version 1.1 remains public while this revision awaits Eric's approval. The original native review contributions are preserved; those reviewers are not represented as having reviewed the new wording.

September 13: bounded Bot review and editor's 1.3 revision

A Grok Bot native contribution reviewed the frozen 1.2 brief, memo, methods and two context files. It returned 997 words. Backend model and reasoning setting were not exposed; it is not credited as Grok Heavy or a new independent provider. The final reports all five source sections and end markers; a follow-up verified the saved packet's bytes/hash and markers. That does not independently establish comprehension. No new X search, experiment or deployed test occurred in this review.

The editor accepted its corrections to the opening, personal pronoun and owner-experience wording, and adapted its suggestion to clarify the nested ask without turning the short brief into a list of thread participants. The added practical questions combine that review with Eric's earlier questions about environment, AI roles, open models, incentives, interpretability and future capabilities. They are design proposals with explicit limits. At this stage the exact 1.3 additions had not yet received the subsequent delta review recorded below. Earlier reviewers are not represented as having approved wording they did not see.

September 13: scheduled review of the 1.3 additions

A further Grok Bot contribution reviewed the frozen 1.3 brief, memo, review record and editorial disposition, plus the 1.2 memo and two context records. Its 778-word final reported all seven source blocks and the packet end marker. The source packet and final bytes were independently checked. Backend model and effort were not exposed, so this is not credited as Grok Heavy or an additional independent model family. It made no live X search or experimental tests.

The review arrived through the normal scheduled Board route while the Bot desktop client remained closed. That tests this scoped contribution's delivery, not correctness of its recommendations, whole-host outage recovery or a native-only architecture: the route still used a custom helper. The claim and final appeared 78 seconds apart. This timing is not a general service guarantee.

The editor retained the short brief as the primary handoff, labelled the practical questions an optional technical annex, and made the coverage ambition explicit without supplying an unsupported percentage. A separate internal check clarified that a second model does not by itself establish evaluator independence. These small post-review edits were checked editorially; the native reviewer did not see their exact final wording. The illustrative workflow and nuanced interpretability limitations remain in the optional annex. The review's suggestion that a diagram itself implied a shipped product was not accepted. Its statement that the entire brief was new was unsupported: a brief existed in 1.2.

The delivery record and qualified disposition are available in [the project review record](#). No additional complete multi-provider round, live reply-tree search, pilot, funding or endorsement is implied. Public version 1.1 and the private approval site's older 1.2 copy remain unchanged until separately updated.